Structured Streaming and Auto Loader

Common Patterns. Clear Fixes.

50 engineering patterns covering Auto Loader, Structured Streaming fundamentals, watermarks and late data, checkpointing, triggers and output modes, stateful operations, and streaming to Delta in Databricks. Patterns every streaming engineer must recognise.

AUTHOR

Sanjay Chandra

50

SCENARIOS

INTRODUCTION

Streaming looks like batch until something goes wrong.

Structured Streaming is one of the most powerful features in Databricks. It is also one of the most misunderstood. Engineers who move from batch to streaming bring their batch intuitions with them: if the job ran without errors the data is correct, restarting a job reruns it from where it left off, and more workers means faster processing. None of these intuitions reliably transfer to streaming.

This book covers seven chapters. Chapter 1 covers Auto Loader and the specific ways file ingestion fails silently or expensively when checkpoint management, schema inference, and notification modes on ADLS Gen2 are not understood. Chapter 2 covers Structured Streaming fundamentals including the failure modes that only appear after extended runtime. Chapter 3 covers watermarks and late data, where misconfiguration produces incorrect aggregations that are not visible until business users notice missing windows. Chapter 4 covers checkpointing, including how checkpoint moves, deletions, and runtime upgrades silently produce data loss or full reprocessing. Chapter 5 covers triggers and output modes and the cost and correctness failures that appear when the wrong choice is made. Chapter 6 covers stateful operations and the ways accumulated state breaks streaming jobs after weeks or months of runtime. Chapter 7 covers streaming to Delta and the maintenance patterns specific to using Delta as a continuous sink.

Every scenario follows the same structure: the situation, the wrong thinking, what is actually happening, and the fix. Streaming problems are particularly dangerous because they often appear only after extended runtime. A watermark configuration that drops late data may not be noticed until a business report is audited weeks after the data was lost.

01

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 4

Chapter 1
Auto Loader

Eight scenarios covering the specific ways Auto Loader fails silently, processes files multiple times, or accumulates costs when checkpoint management, schema inference, and notification modes on ADLS Gen2 are not understood before deployment.

01

AUTO LOADER

Auto Loader processed the same file twice. Duplicate rows appeared downstream.

THE SITUATION

An Auto Loader job processes files from cloud storage into a Delta table. After a job restart following a failure, duplicate rows appear in the target table for files that were processed before the failure. The Auto Loader job shows no errors on the restart run.

WRONG THINKING

Engineers assume Auto Loader provides exactly-once file processing guarantees regardless of job failure and restart. A restarted Auto Loader job is expected to resume from where it left off and skip files already processed.

WHAT IS ACTUALLY HAPPENING

Auto Loader uses a checkpoint directory to track which files have been processed. Exactly-once guarantees depend on the checkpoint being consistent with the state of the output Delta table. If the job failed after Auto Loader committed a file to its checkpoint but before the corresponding Delta write transaction was committed, the checkpoint and the target table are inconsistent. On restart, Auto Loader skips the file because it is marked as processed in the checkpoint, but the Delta table does not have the data. Conversely if the Delta write committed but the checkpoint did not update before the failure, Auto Loader reprocesses the file on restart, producing duplicates.

THE FIX

Use `foreachBatch` with idempotent writes to make Auto Loader processing exactly-once end-to-end. Write to Delta using MERGE rather than append so that reprocessed files produce upserts rather than duplicate inserts. Alternatively use Delta's built-in idempotent write support by specifying a transaction identifier in the write operation. Ensure the checkpoint directory and the target Delta table are treated as a single atomic unit and test failure and restart behavior explicitly before going to production.

02

AUTO LOADER

Your Auto Loader job ran for hours with no new files. CPU was pegged at 100 percent.

THE SITUATION

An Auto Loader job is configured in continuous mode. No new files have arrived in the monitored location for several hours. The cluster shows 100 percent CPU utilization despite no data being processed. Cluster costs are accumulating with zero productive work.

WRONG THINKING

Engineers run Auto Loader in continuous mode expecting it to idle efficiently when no files are available. Continuous mode is assumed to be similar to a long-running streaming job that sleeps between micro-batches when there is no input.

WHAT IS ACTUALLY HAPPENING

When Auto Loader is configured in directory listing mode without file notification, it polls the source directory continuously to check for new files. Even when no new files are present, the polling operation reads the directory listing from cloud storage on every check interval. For large directories with millions of existing files, this listing operation is CPU-intensive because Auto Loader must compare the current directory listing against its internal state to identify new files. The CPU usage is from the continuous directory comparison, not from data processing.

THE FIX

Switch Auto Loader from directory listing mode to file notification mode using `cloudFiles.useNotifications = true`. On Azure this provisions an Event Grid subscription on the ADLS Gen2 container and a Storage Queue that Auto Loader consumes from, eliminating the need for continuous directory polling. The Event Grid subscription listens for the `FlushWithClose` event so that only fully-committed files trigger ingestion. This reduces CPU usage to near zero during periods with no new files. For directories that already contain millions of files at the time of the switch, limit the initial backfill volume with `cloudFiles.maxFilesPerTrigger`. On Databricks Runtime 14.3 LTS and above, prefer Auto Loader with file events on a Unity Catalog external location, which scopes notifications to the relevant volume rather than the whole storage account.

03

AUTO LOADER

Auto Loader detected new files. They were not processed until the next trigger.

THE SITUATION

An Auto Loader job is configured with a fixed interval trigger of 1 hour. New files arrive every few minutes throughout the day. Files arriving between trigger intervals wait up to 1 hour before being processed. Data freshness is significantly worse than expected.

WRONG THINKING

Engineers configure Auto Loader with a 1-hour trigger interval because the downstream consumer only needs hourly updates. The assumption is that Auto Loader will process files as they arrive and the trigger interval only affects when results are written downstream.

WHAT IS ACTUALLY HAPPENING

Auto Loader's trigger interval controls when micro-batches execute and when new files are discovered and processed. Files that arrive between trigger intervals are not processed until the next trigger fires. A 1-hour trigger means all files arriving in a given hour are processed together in the next hourly micro-batch. The data freshness is determined by the trigger interval, not by the file arrival time. Files arriving at 9:01am in a 1-hour trigger job are not processed until the 10:00am trigger fires.

THE FIX

Align the trigger interval with the actual data freshness requirement. If the downstream consumer needs hourly data, a 1-hour trigger is appropriate. If data freshness should be closer to file arrival time, reduce the trigger interval to match. For near-real-time requirements use `trigger(availableNow=True)` in a frequently scheduled job or configure a continuous trigger. Balance trigger frequency against cluster cost, since more frequent triggers increase micro-batch overhead.

04

AUTO LOADER

You switched Auto Loader from directory listing to file notification. Files were missed.

THE SITUATION

An Auto Loader job is changed from directory listing mode to file notification mode to reduce polling overhead. After the switch, some files that arrived during the transition period are not processed. The files exist in the ADLS Gen2 storage location but do not appear in the output.

WRONG THINKING

Engineers switch notification modes and assume the transition is seamless. The checkpoint is preserved so Auto Loader is expected to continue from where it left off without missing any files.

WHAT IS ACTUALLY HAPPENING

When switching Auto Loader from directory listing to file notification mode on Azure, the notification infrastructure (an Event Grid subscription on the storage account and a Storage Queue that Auto Loader reads from) must be created and become active before file events will be captured. During the period between when the old directory-listing job stops and when the Event Grid subscription is active and emitting events into the queue, files that arrive in the ADLS Gen2 container may not generate notification events because the subscription was not yet listening. These files are never placed in the queue and Auto Loader in notification mode will never discover them. A related but separate failure mode is when files are written to ADLS Gen2 via the Create File API or streamed writes: the `BlobCreated` event can fire before the file is fully committed, and if the Event Grid filter is not set to `FlushWithClose`, Auto Loader sees empty files and ignores them.

THE FIX

During any Auto Loader mode transition, run a directory listing scan after switching to notification mode to identify and process any files that arrived during the transition gap. Use a one-time batch job to scan the source directory and process files whose names do not appear in the Auto Loader checkpoint. Enable `cloudFiles.backfillInterval` (for example, 1 day) so Auto Loader periodically reconciles the queue against the actual storage contents and catches any events lost in transit. Verify the Event Grid subscription is filtering on `FlushWithClose` rather than the broader `BlobCreated` if writers use streamed or multi-part uploads. Always test mode transitions in a staging environment with a controlled file set before applying to production.

05

AUTO LOADER

Your Auto Loader checkpoint grew to 50GB. Startup time increased dramatically.

THE SITUATION

An Auto Loader job has been running for 8 months ingesting millions of files. The checkpoint directory has grown to 50GB. Job startup time, which previously took under a minute, now takes 15 minutes as Auto Loader reads and processes the checkpoint before beginning to ingest new files.

WRONG THINKING

Engineers assume the Auto Loader checkpoint is a small metadata file that records which files have been processed. As the checkpoint grows with more files processed, its impact on startup time is not monitored.

WHAT IS ACTUALLY HAPPENING

Auto Loader's checkpoint contains metadata for every file it has processed, including file names, sizes, modification times, and processing status. For a job that has processed millions of files over months, this checkpoint can grow to many gigabytes. On startup Auto Loader must read and parse the checkpoint to determine which files have already been processed and which are new. Reading a 50GB checkpoint requires significant I/O and memory, extending startup time proportionally with checkpoint size.

THE FIX

Implement checkpoint compaction by periodically creating a new Auto Loader job with a fresh checkpoint that starts from the current watermark position rather than the beginning of history. For large long-running Auto Loader jobs, evaluate whether archiving or deleting source files that have been successfully processed and verified reduces the checkpoint scope. Monitor checkpoint directory size as a key operational metric and establish a threshold above which checkpoint maintenance is triggered. Databricks Auto Loader manages some checkpoint compaction automatically in newer DBR versions but monitoring growth remains important.

06

AUTO LOADER

Auto Loader silently skipped malformed files. No error was raised.

THE SITUATION

An Auto Loader job ingests JSON files from cloud storage. Several files with malformed JSON syntax are placed in the source directory. The job continues running without errors. The malformed files do not appear in the output but no error or alert is generated. The files sit in the source directory unprocessed.

WRONG THINKING

Engineers assume Auto Loader will raise an error when it encounters files it cannot parse. A job that runs without errors is assumed to have successfully processed all files in the source directory.

WHAT IS ACTUALLY HAPPENING

Auto Loader uses the underlying Spark JSON or CSV parser, and that parser defaults to `PERMISSIVE` mode. In PERMISSIVE mode, malformed records are placed in the `_corrupt_record` column rather than raising an exception. If the output schema does not include `_corrupt_record` as a string column, those rows are silently dropped on read. Auto Loader independently marks the file as processed in its checkpoint as long as the parser did not raise, so even when all rows from a file were dropped due to parse errors the file is treated as done and is never retried. There is no native alert for dropped rows; the only signal is the row count gap between source and sink.

THE FIX

There are two separate mechanisms to distinguish here. Auto Loader's `cloudFiles.schemaEvolutionMode = rescue` handles schema-level issues: when an incoming field does not match the expected schema or has a type mismatch, the unexpected data is captured in the `_rescued_data` column rather than being silently dropped. This is the right setting for catching unexpected fields and type drift. For records that are completely unparseable (broken JSON syntax, truncated lines, encoding errors), use the parser-level `mode` option instead: set `mode = FAILFAST` to raise an exception on the first malformed record and fail the micro-batch, making the issue immediately visible. Alternatively use `mode = PERMISSIVE` with `columnNameOfCorruptRecord` set and explicitly included in the schema as a string column, then monitor that column for non-null values after each micro-batch. Combine both: enable rescue mode for schema drift and PERMISSIVE with an explicit corrupt-record column for parse errors, alongside a row count check that compares processed rows against expected volume based on file sizes.

07

AUTO LOADER

You deleted the Auto Loader checkpoint. All files were reprocessed from scratch.

THE SITUATION

A data engineer deletes the Auto Loader checkpoint directory to resolve what they believe is a checkpoint corruption issue. On the next run Auto Loader treats all files in the source directory as new and reprocesses the entire file history, producing millions of duplicate rows in the target Delta table.

WRONG THINKING

Engineers treat the checkpoint as a cache that can be cleared safely. Deleting the checkpoint is assumed to reset Auto Loader to a clean state from which it will process only new files going forward. The relationship between the checkpoint and which files are considered new is not understood.

WHAT IS ACTUALLY HAPPENING

The Auto Loader checkpoint is the only record of which files have already been processed. Without the checkpoint, Auto Loader has no way to distinguish files that were processed previously from genuinely new files. On restart with no checkpoint, Auto Loader scans the entire source directory and considers every file it finds as unprocessed. For a source directory containing years of file history, this means processing all historical files from the beginning.

THE FIX

Never delete an Auto Loader checkpoint without first archiving or removing the source files that have already been processed. If checkpoint corruption is suspected, attempt checkpoint repair rather than deletion. Use Databricks support to diagnose checkpoint corruption issues before taking destructive action. For disaster recovery planning, implement a process that records the last processed file timestamp or version in a separate durable store so that a fresh checkpoint can be created that starts from the correct historical position rather than the beginning of all time.

08

AUTO LOADER

Auto Loader is reading files correctly. Schema inference is wrong on every run.

THE SITUATION

An Auto Loader job uses schema inference to automatically detect the schema of incoming JSON files. On each run the inferred schema differs slightly from the previous run depending on which files happen to be sampled for inference. Downstream transformations that depend on specific column names or types break intermittently.

WRONG THINKING

Engineers enable schema inference expecting it to provide a consistent schema based on the file contents. Schema inference is assumed to converge on a stable schema after the first few runs as the system learns the data structure.

WHAT IS ACTUALLY HAPPENING

Auto Loader schema inference samples a subset of files to determine the schema. The sample is taken from files available at the time of inference, which varies between runs. If different files in the source directory have slightly different schemas such as different optional fields or varying data types for the same column, the inferred schema changes depending on which files are included in the sample. Schema inference is not deterministic across runs when the source data has schema variability.

THE FIX

Define an explicit schema rather than relying on automatic inference for production Auto Loader jobs. Use `spark.readStream.schema(defined_schema)` to provide a fixed schema that all incoming files must conform to. Enable schema evolution handling for intentional changes using `cloudFiles.schemaEvolutionMode`. Run schema inference once in development to capture the schema, then hardcode it in the production job. This eliminates inference variability and makes schema changes explicit and controlled.

02

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 4

Chapter 2 Streaming Fundamentals

Eight scenarios covering the streaming failure modes that only appear after extended runtime, including memory growth, duplicate output, and the specific ways restarting a streaming job does not behave as engineers expect.

09

STREAMING FUNDAMENTALS

Your streaming job shows 0 rows processed for 20 minutes then processes everything at once.

THE SITUATION

A Structured Streaming job monitors a Delta table source. For 20 minutes after startup the job shows 0 rows processed per micro-batch. Then in a single micro-batch it processes all 500,000 rows that arrived during the 20-minute window at once.

WRONG THINKING

Engineers expect streaming to process rows as they arrive. Zero rows per micro-batch for 20 minutes indicates the stream is not reading from the source. A sudden bulk processing of all buffered rows is unexpected.

WHAT IS ACTUALLY HAPPENING

When reading from a Delta table source, Structured Streaming uses the Delta transaction log to identify new data versions. If the Delta source table is being written to by a batch job that runs every 20 minutes, the Delta version only advances when the batch completes and commits. Between batch commits, no new Delta versions exist and the streaming job finds no new data. When the batch commits after 20 minutes, the streaming job sees a new Delta version with all rows from that batch and processes them in a single micro-batch.

THE FIX

Align expectations for streaming latency with the commit frequency of the source. If the source is a Delta table written by a 20-minute batch job, the effective streaming latency is 20 minutes regardless of the streaming trigger interval. For lower latency, the source must write more frequently. Use Delta table streaming with `maxBytesPerTrigger` or `maxFilesPerTrigger` to limit the size of each micro-batch when large commits arrive, preventing a single large micro-batch from causing downstream latency spikes.

10

STREAMING FUNDAMENTALS

Your streaming query runs correctly. The output table has duplicate rows.

THE SITUATION

A Structured Streaming job writes aggregated results to a Delta output table. The aggregations appear correct but the output table has duplicate rows for some key values. The streaming job has been running for several days without being restarted.

WRONG THINKING

Engineers assume Structured Streaming provides exactly-once output semantics for all write operations. Since the job has not been restarted, duplicates cannot be caused by reprocessing. The streaming job is assumed to produce each output row exactly once.

WHAT IS ACTUALLY HAPPENING

Structured Streaming provides exactly-once processing guarantees only for sinks that support idempotent writes or transactional commits. For Delta Lake sinks, exactly-once is supported when using append mode with checkpointing or when using foreachBatch with idempotent write logic. If the streaming query uses append mode and a micro-batch write fails and retries, the retry may append the same rows again. Additionally if the streaming job writes to a non-Delta sink without idempotency guarantees, duplicate rows can accumulate over time from internal Spark retry mechanisms.

THE FIX

For streaming aggregations written to Delta, use complete or update output mode rather than append mode for stateful aggregations to avoid accumulating duplicate partial results. Use foreachBatch with MERGE to implement idempotent upsert logic that handles reprocessed micro-batches correctly. Verify the output mode is appropriate for the aggregation type and that the Delta sink is configured to support the streaming semantics required.

11

STREAMING FUNDAMENTALS

You restarted a streaming job. It reprocessed all historical data from the source.

THE SITUATION

A Structured Streaming job is restarted after being stopped for maintenance. Instead of resuming from the last processed position, the job starts from the beginning of the source data and reprocesses all historical records.

WRONG THINKING

Engineers assume a restarted streaming job automatically resumes from where it stopped, similar to a batch job that failed and retried. The checkpoint is assumed to preserve the job's position automatically.

WHAT IS ACTUALLY HAPPENING

Structured Streaming uses a checkpoint to persist the job's current processing position. On restart, the job reads the checkpoint to determine where to resume. If the checkpoint directory is not specified or the specified path does not exist or was deleted, the job starts from the default starting position which is typically the beginning of available data. If the checkpoint path was changed between the stop and restart, the job uses the new empty checkpoint and treats all data as new.

THE FIX

Always specify an explicit checkpoint location using `.option('checkpointLocation', '/path/to/checkpoint')` in the streaming write configuration. Verify the checkpoint path is consistent between the stopped and restarted job by checking the streaming query definition before starting. Store checkpoint directories in durable cloud storage rather than local disk. Before restarting a streaming job after any code or configuration change, verify that the checkpoint is compatible with the new query definition.

12

STREAMING FUNDAMENTALS

Your streaming job is running. Input rate is 1000 rows per second. Processing rate is 10.

THE SITUATION

A Structured Streaming job receives 1000 rows per second from a Kafka source. The Spark UI shows a processing rate of 10 rows per second. The input queue is growing continuously. The job is falling progressively further behind the real-time data.

WRONG THINKING

Engineers assume that a running streaming job is processing data at approximately the input rate. A running status means the job is keeping up with the stream. The growing lag is discovered when downstream data freshness degrades.

WHAT IS ACTUALLY HAPPENING

Structured Streaming processes data in micro-batches. If each micro-batch takes longer to process than the interval between incoming batches, the job falls behind. A 100x mismatch between input rate and processing rate indicates the processing logic is severely under-resourced or inefficient. Common causes include complex transformations that do not scale with data volume, too few executors, stateful operations with large state sizes, or expensive external calls inside the streaming logic.

THE FIX

Monitor both the input rate and the processing rate in the Spark Streaming UI and set an alert when processing rate falls below input rate for more than a few minutes. For a persistent 100x lag, investigate the bottleneck: check executor count versus parallelism, look for skewed partitions in the Spark UI, profile the transformation logic for expensive operations such as UDFs or external lookups, and evaluate whether the state size has grown to a point where stateful operations are slow. Scale the cluster to match the processing requirement or reduce the complexity of the streaming logic.

13

STREAMING FUNDAMENTALS

You stopped a streaming job cleanly. The checkpoint shows it is still running.

THE SITUATION

A Structured Streaming job is stopped using the Databricks workflow stop button. The Spark UI shows the streaming query as terminated. The checkpoint directory still contains a lock file indicating the query is active. Attempting to restart the job fails with a checkpoint lock error.

WRONG THINKING

Engineers assume a cleanly stopped streaming job releases all checkpoint locks before terminating. A stopped job should leave the checkpoint in a clean state ready for the next start. The checkpoint lock error on restart is unexpected and assumed to be a bug.

WHAT IS ACTUALLY HAPPENING

Structured Streaming acquires a lock on the checkpoint directory to prevent multiple concurrent instances of the same query from writing conflicting state. When a streaming job is stopped, the lock should be released as part of the shutdown process. If the shutdown is not clean, for example if the driver process is killed rather than shut down gracefully, the lock file may not be removed. The next attempt to start the query finds an existing lock and refuses to proceed to prevent data corruption.

THE FIX

Delete the lock file from the checkpoint directory manually when a stale lock is confirmed. The lock file is typically located at `checkpoint_path/sources/0/` or a similar path within the checkpoint structure. Verify the previous job instance is fully terminated before deleting the lock. After deleting the lock, restart the job. To prevent stale locks, prefer graceful shutdown mechanisms over forceful process termination. In Databricks, use the workflow cancel option rather than cluster termination when stopping streaming jobs.

14

STREAMING FUNDAMENTALS

Your streaming job processes data in order. Output rows are out of order.

THE SITUATION

A Structured Streaming job reads events from Kafka in order and writes them to a Delta output table. Downstream queries against the output table return rows in an order that does not match the original event sequence. Events that arrived later in the Kafka topic appear earlier in the output table.

WRONG THINKING

Engineers assume that a streaming job that reads data in order will write data in order. Since Kafka preserves message order within a partition and the streaming job reads from Kafka sequentially, the output is expected to reflect the input order.

WHAT IS ACTUALLY HAPPENING

Structured Streaming processes micro-batches with internal Spark parallelism. Within each micro-batch, multiple partitions are processed concurrently across multiple executors. The order in which processed rows are written to the output Delta table is determined by which executor finishes first, not by the original event order. Additionally Delta Lake write operations commit atomically but the physical file layout does not preserve row insertion order. Output tables have no guaranteed row ordering unless ORDER BY is specified at query time.

THE FIX

Do not rely on physical row insertion order for event sequence queries. Use the event timestamp or sequence number from the original event source as the ordering column and include it in the output table. When querying the output table for ordered results, always include an ORDER BY clause on the event timestamp or sequence field. Design downstream consumers to handle out-of-order data from streaming outputs as a fundamental characteristic of parallel stream processing.

15

STREAMING FUNDAMENTALS

You changed the streaming query logic. The checkpoint is incompatible with the new query.

THE SITUATION

A Structured Streaming job is updated with a new transformation that adds an additional aggregation column. When the job is restarted with the updated query and the existing checkpoint, it fails with a checkpoint incompatibility error. The existing checkpoint cannot be used with the modified query.

WRONG THINKING

Engineers make what appears to be a minor query change and expect the job to restart normally using the existing checkpoint. The checkpoint is assumed to store only position information that is independent of the query logic.

WHAT IS ACTUALLY HAPPENING

Structured Streaming checkpoints store more than position information. They also store the query plan, state schema, and other query-specific metadata that must be consistent between the checkpoint and the running query. Changes that modify the query plan, add or remove stateful operations, change aggregation keys, or alter the output schema can make the existing checkpoint incompatible with the new query. Spark detects this incompatibility and refuses to use the checkpoint to prevent corrupted state from being applied to the modified query.

THE FIX

For query changes that are incompatible with the existing checkpoint, start the job with a new checkpoint directory. Set `.option('checkpointLocation', '/new/checkpoint/path')` in the updated query. The job will start from the beginning of available data or from the configured starting position. To avoid reprocessing all historical data, use `startingOffsets` or `startingVersion` to specify a starting position that skips historical data already processed by the previous version of the job. Test query changes for checkpoint compatibility before deploying to production by checking the Spark documentation for which query changes break checkpoint compatibility.

16

STREAMING FUNDAMENTALS

Your streaming job has been running for 30 days. Memory usage keeps growing.

THE SITUATION

A Structured Streaming job has been running continuously for 30 days without restart. Over time memory usage on the driver and executors has increased steadily. The job is now running near the memory limit and is at risk of OOM failure.

WRONG THINKING

Engineers expect a long-running streaming job to have stable memory usage once it reaches steady state. A job that has been running successfully for weeks is assumed to be healthy. Gradual memory growth is not monitored as a key metric.

WHAT IS ACTUALLY HAPPENING

Structured Streaming accumulates state for stateful operations such as aggregations, joins, and deduplication. Even with watermarks configured to bound state size, small amounts of state metadata and internal bookkeeping structures can accumulate over time. Additionally Spark's JVM process accumulates garbage that is not fully collected during normal GC cycles. Over 30 days of continuous operation, the cumulative effect of small memory leaks in stateful logic, un-GC'd JVM objects, and growing internal metadata can push memory usage to critical levels.

THE FIX

Implement a planned restart schedule for long-running streaming jobs. A weekly or bi-weekly restart during a low-traffic window reclaims accumulated memory, applies any pending software updates, and resets internal state metadata. Structured Streaming restarts from the checkpoint so data continuity is preserved. Monitor memory usage trends over time and set alerts when memory usage exceeds 80 percent of available heap. Review all stateful operations for correctness of watermark and state TTL configuration to minimize accumulation.

03

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 4

Chapter 3 Watermarks and Late Data

Seven scenarios covering how watermark configuration determines which late events are included and which are silently dropped, and the specific ways watermark misconfiguration produces incorrect aggregations.

17

WATERMARKS AND LATE DATA

You set a 1 hour watermark. Data arriving 59 minutes late is being dropped.

THE SITUATION

A streaming aggregation uses a 1-hour watermark to handle late arriving events. Events arrive up to 45 minutes late under normal conditions. During a brief upstream delay, events arrive 59 minutes late. These events are dropped from the aggregation output. The downstream report shows gaps for the affected time windows.

WRONG THINKING

Engineers set a 1-hour watermark expecting all events arriving within 1 hour of their event time to be included. 59 minutes is less than 1 hour so those events should be included in the aggregation.

WHAT IS ACTUALLY HAPPENING

Watermark behavior in Structured Streaming defines the maximum lateness for event inclusion based on the difference between the current watermark value and the event timestamp. The watermark advances as new events arrive with later event times. An event is considered late if its event timestamp falls below the current watermark value. The current watermark is the maximum observed event time minus the watermark delay. If events with the latest timestamps have advanced the watermark to T-1hour, an event with timestamp T-61minutes is dropped because it falls below the current watermark even though it arrived within the 1-hour window. The watermark boundary is relative to the current maximum event time, not to the current wall clock time.

THE FIX

Set the watermark duration generously enough to accommodate the maximum observed late arrival plus a safety buffer. If events arrive up to 45 minutes late under normal conditions and up to 59 minutes during upstream delays, a 90-minute watermark provides coverage for normal delays and some buffer for unusual delays. Monitor the distribution of late arrival times in production and set the watermark to the 99th percentile of late arrival time plus a buffer rather than the maximum observed case.

18

WATERMARKS AND LATE DATA

Your watermark is advancing too fast. Late arriving events are missing from aggregations.

THE SITUATION

A streaming aggregation with a 30-minute watermark is producing incorrect results. Events that arrive 20 minutes after their event time are being dropped even though the watermark should accommodate 30-minute late arrivals. Some time windows show complete results while others have missing events.

WRONG THINKING

Engineers set a 30-minute watermark and expect all events within 30 minutes of their event time to be included. Events arriving 20 minutes late should be within the watermark. The inconsistency between windows that include late events and windows that drop them is attributed to a Spark bug.

WHAT IS ACTUALLY HAPPENING

The watermark advances based on the maximum event timestamp observed across all partitions. If one Kafka partition or data source delivers events with very recent timestamps while another partition has delayed delivery, the maximum event timestamp is determined by the fast partition. The watermark may advance to a point where events from the slow partition, which have older event timestamps, appear to be late and are dropped. The watermark sees the maximum event time across all partitions simultaneously, so fast partitions can advance the watermark beyond what slow partitions can deliver.

THE FIX

When streaming from partitioned sources such as Kafka with multiple partitions, set the watermark to account for the maximum skew between the fastest and slowest partition, not just the maximum late arrival time for individual events. Monitor per-partition event time lag in the Spark Streaming UI. If partition skew is significant, either increase the watermark to accommodate the skew or implement partition-level event time monitoring and alerting to detect when partitions fall behind.

19

WATERMARKS AND LATE DATA

You increased the watermark window to 24 hours. State size exploded.

THE SITUATION

A streaming aggregation was experiencing dropped late events. The watermark was increased from 1 hour to 24 hours to ensure all late events are captured. Shortly after the change, the streaming job begins running out of memory. State size has grown by 24 times.

WRONG THINKING

Engineers increase the watermark to capture more late events, expecting the change to have minimal impact on performance. A longer watermark window means more events are captured, which is the desired outcome.

WHAT IS ACTUALLY HAPPENING

The watermark duration directly determines how long Spark must retain aggregation state for each time window. With a 1-hour watermark, Spark retains state for time windows that are up to 1 hour old. With a 24-hour watermark, Spark retains state for time windows up to 24 hours old. For a streaming aggregation that maintains state per minute, a 24-hour watermark requires retaining state for 1440 minutes worth of windows simultaneously rather than 60. For a high-throughput stream with many distinct grouping keys, this 24x increase in retained state windows translates directly to a 24x increase in state memory requirements.

THE FIX

Before increasing a watermark, estimate the state size impact by calculating the number of distinct aggregation windows that will be retained. For a high-cardinality stream, the state size is proportional to the number of distinct keys multiplied by the number of retained time windows. If a 24-hour watermark is necessary, ensure the cluster has sufficient memory to support the increased state size. Consider whether a coarser time window granularity such as hourly rather than per-minute aggregation can reduce the number of distinct state entries while still meeting the business requirement.

20

WATERMARKS AND LATE DATA

Your watermark is set correctly. Some late data is still being included past the threshold.

THE SITUATION

A streaming aggregation has a 1-hour watermark. Events arriving more than 1 hour late should be dropped. Some events arriving 90 minutes late are appearing in the aggregation output, violating the expected late data policy.

WRONG THINKING

Engineers set the watermark and expect it to precisely enforce the late data boundary. Any event arriving after the watermark threshold should be dropped. Events that appear in the output despite being outside the threshold are assumed to indicate a watermark calculation error.

WHAT IS ACTUALLY HAPPENING

Watermark enforcement in Structured Streaming is eventual rather than immediate. The watermark advances based on observed event times in micro-batches. If a micro-batch does not contain events with timestamps newer than the current maximum, the watermark does not advance and the late data boundary does not move. During periods of low traffic when few new events arrive with recent timestamps, the watermark may lag behind the actual current time. In these periods, events that arrive with timestamps that would be considered late based on wall clock time may still be within the current watermark boundary because the watermark has not advanced to catch up with the current time.

THE FIX

Understand that watermark-based late data filtering is relative to the observed event time stream rather than wall clock time. The watermark does not advance based on time passing but based on new events arriving with later timestamps. For use cases requiring strict wall clock-based late data enforcement, implement an additional filter at output time that drops rows with event timestamps older than the current wall clock time minus the maximum late arrival tolerance. This provides a deterministic late data boundary independent of watermark advancement rate.

21

WATERMARKS AND LATE DATA

Event time aggregations are correct. Processing time aggregations are wrong.

THE SITUATION

A streaming job performs two aggregations: one on event time and one on processing time. The event time aggregation produces correct results validated against the source data. The processing time aggregation produces counts that differ significantly from expected values.

WRONG THINKING

Engineers validate the event time aggregation and confirm it is correct. Since event time aggregation is the more complex operation and it is correct, processing time aggregation is expected to be correct by extension.

WHAT IS ACTUALLY HAPPENING

Event time and processing time aggregations have fundamentally different semantics. Event time aggregations group events by when they occurred according to the event's own timestamp field. Processing time aggregations group events by when Spark processed them. If a micro-batch processes a large backlog of old events, all of those events fall into the current processing time window even though their event times span a long historical period. The processing time window shows an inflated count because it reflects when Spark caught up with the backlog, not when the events originally occurred.

THE FIX

Use event time aggregations for all business-facing metrics that should reflect when events occurred. Processing time aggregations are appropriate for operational metrics about streaming job throughput and latency, not for business data analysis. When investigating discrepancies, compare event time distribution against processing time distribution to identify whether the mismatch is caused by backlog processing or genuine data arrival patterns.

22

WATERMARKS AND LATE DATA

Your watermark never advances. The streaming job holds all state indefinitely.

THE SITUATION

A Structured Streaming aggregation job has been running for several days. Memory usage is growing continuously. Investigation reveals the watermark has not advanced since the job started. All aggregation state from every micro-batch is being retained because the watermark boundary never moves.

WRONG THINKING

Engineers configure a watermark and assume it will advance automatically as new events arrive. The watermark is expected to limit state retention by allowing completed time windows to be cleaned up.

WHAT IS ACTUALLY HAPPENING

Watermarks advance based on the maximum event timestamp observed across the stream. If the event timestamp column contains null values, constant values, or is incorrectly mapped to a non-timestamp column, Spark cannot advance the watermark because it cannot extract a meaningful event time. A watermark that does not advance never triggers state cleanup. All aggregation state is retained indefinitely, causing continuous memory growth.

THE FIX

Verify the watermark column is correctly specified and contains valid timestamp values. Use `df.select(col('event_time')).describe().show()` on a sample of the stream to confirm the column contains non-null, valid timestamps. Add a data quality check on the event time column at stream ingestion that alerts when null or invalid timestamps are detected. If null timestamps are expected in some records, handle them explicitly before the watermark aggregation either by filtering them out or by replacing them with a default timestamp that does not affect the watermark advancement.

23

WATERMARKS AND LATE DATA

You have multiple streaming sources with different latencies. The watermark is blocked.

THE SITUATION

A streaming job joins two streams. Stream A delivers events with low latency. Stream B delivers events with 30-minute latency. A watermark is configured on the joined stream. The watermark advances very slowly despite Stream A providing frequent high-timestamp events.

WRONG THINKING

Engineers configure a watermark on the joined stream expecting it to advance based on the events from both streams. Stream A provides frequent recent timestamps which should drive the watermark forward.

WHAT IS ACTUALLY HAPPENING

In a stream-stream join, Spark must maintain state for both streams to perform the join. The watermark for the joined stream advances based on the minimum of the maximum event times seen across all joined streams. Since Stream B has 30-minute latency, its maximum event time is always 30 minutes behind Stream A's maximum event time. The join watermark can only advance as fast as the slowest stream. Even though Stream A provides recent timestamps continuously, the watermark is held back by Stream B's delayed timestamps. State for both streams is retained until the join watermark advances.

THE FIX

Apply separate watermarks to each stream before the join rather than a single watermark on the joined stream. This allows each stream's state to be cleaned up based on its own event time progression. For stream-stream joins with highly asymmetric latencies, consider whether a stream-static join is more appropriate, where the slower stream is treated as a static lookup table refreshed on a schedule rather than a streaming source that participates in the watermark calculation.

04

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 4

Chapter 4 Checkpointing

Seven scenarios covering checkpoint management, the operational complexity of moving and upgrading checkpoints, and the specific ways checkpoint failures lead to data loss or reprocessing.

24

CHECKPOINTING

You moved the checkpoint to a new location. The job reprocessed everything.

THE SITUATION

A streaming job checkpoint is moved to a new cloud storage path as part of a storage reorganisation. The job is restarted pointing to the new checkpoint path. Instead of resuming from the last processed position, the job processes all historical data from the beginning of the source.

WRONG THINKING

Engineers move the checkpoint directory contents to the new path and update the job configuration. Since the checkpoint files are present at the new path, the job is expected to resume normally.

WHAT IS ACTUALLY HAPPENING

Structured Streaming checkpoint directories contain multiple subdirectories with specific internal structures including commits, offsets, sources, and state directories. Simply moving the files to a new path preserves the file contents but may not preserve the relative directory structure that Spark expects. If any subdirectory is missing or incorrectly structured at the new path, Spark treats the checkpoint as absent or corrupt and starts a new streaming session from the beginning of the source data.

THE FIX

Use cloud storage copy operations that preserve full directory structure rather than move operations that may flatten or alter relative paths. After copying, verify the complete directory structure at the new location matches the original exactly using a recursive directory comparison. Test the checkpoint by starting the job with a dry run mode before pointing production traffic to it. Always maintain the original checkpoint as a backup until the moved checkpoint is confirmed working.

25

CHECKPOINTING

Two streaming jobs share a checkpoint directory. Data is corrupted.

THE SITUATION

Two instances of a streaming job are started simultaneously, both configured with the same checkpoint path. After running for several hours, the output Delta table contains corrupted aggregation results. Rows appear multiple times with inconsistent values.

WRONG THINKING

Engineers start a second instance of a streaming job to scale processing and configure it to use the same checkpoint as the first instance, expecting both instances to share work and coordinate through the shared checkpoint.

WHAT IS ACTUALLY HAPPENING

Structured Streaming is designed to run as a single query instance per checkpoint. The checkpoint records the query's progress including current offsets, state snapshots, and committed micro-batch identifiers. When two instances write to the same checkpoint simultaneously, each overwrites the other's progress records. Each instance reads offset information written by the other instance and processes data based on those offsets, leading to both instances processing overlapping data ranges and writing conflicting state. The result is corrupted aggregations and duplicate output.

THE FIX

Never share a checkpoint directory between two instances of a streaming job. Each streaming query instance must have an exclusive checkpoint directory. For scaling streaming throughput, increase the number of partitions in the source and the cluster size rather than running multiple job instances. If multiple instances are genuinely needed for different processing logic, each must have a separate checkpoint and process a separate partition range of the source.

26

CHECKPOINTING

Your checkpoint is on local storage. A node failure lost all streaming progress.

THE SITUATION

A streaming job checkpoint is stored on the driver node's local disk. A driver node failure occurs and the node is replaced. The new driver has no checkpoint and the streaming job restarts from the beginning of the source data, reprocessing months of historical events.

WRONG THINKING

Engineers store the checkpoint locally for simplicity and fast I/O. Local storage is faster than cloud storage and the checkpoint seems like a temporary working directory that does not need durability.

WHAT IS ACTUALLY HAPPENING

The streaming checkpoint is the only record of the job's processing position. Local disk storage on a cloud VM is ephemeral. If the VM is replaced, terminated, or fails, all local data including the checkpoint is permanently lost. A streaming job that loses its checkpoint has no way to resume from its previous position and must start over.

THE FIX

Always store streaming checkpoints in durable cloud storage. On Azure Databricks this means an ADLS Gen2 path via the `abfss://` driver, or a Unity Catalog volume backed by ADLS Gen2. Never use local disk paths or paths under `/tmp` for checkpoints; these are on the worker or driver VM's ephemeral storage and are lost when the VM is deallocated. Configure the checkpoint location explicitly using the full storage path rather than a relative path that might resolve to local storage in some environments. Treat the checkpoint as production data that must survive node failures, cluster replacements, and Azure VM deallocation events.

27

CHECKPOINTING

Checkpoint restoration takes 45 minutes every time the job restarts.

THE SITUATION

A Structured Streaming job with complex stateful operations restarts after a planned maintenance window. Checkpoint restoration takes 45 minutes before any new data is processed. During this 45-minute window the streaming job is consuming cluster resources and accumulating lag but producing no output.

WRONG THINKING

Engineers accept the 45-minute restart time as a fixed overhead for a complex streaming job. The checkpoint restoration time is not profiled or optimised because the job runs continuously and restarts are infrequent.

WHAT IS ACTUALLY HAPPENING

Checkpoint restoration time scales with the size of the state stored in the checkpoint. For streaming jobs with large state such as aggregations with many distinct keys, long retention windows, or stream-stream joins, the state snapshot in the checkpoint can be very large. Restoring this state requires reading the entire state snapshot from cloud storage and loading it into executor memory before processing can begin. For very large state, this can take tens of minutes.

THE FIX

Reduce state size by reviewing watermark configurations and ensuring state is being cleaned up as expected. For large-state streaming jobs use the RocksDB state store provider, which uses incremental checkpointing and reduces both checkpoint write time and restore time compared to the in-memory state store. On Databricks Runtime 17.3 and above RocksDB is the default state store provider; on earlier runtimes enable it explicitly with `spark.sql.streaming.stateStore.providerClass = org.apache.spark.sql.execution.streaming.state.RocksDBStateStoreProvider`. Monitor state size as a key metric and reduce aggregation retention windows where business requirements allow.

28

CHECKPOINTING

You upgraded Spark version. The existing checkpoint is incompatible.

THE SITUATION

A Databricks Runtime upgrade changes the underlying Spark version. The streaming job is restarted after the upgrade using the existing checkpoint. The job fails immediately with a checkpoint incompatibility error. The job cannot resume from the existing checkpoint with the new runtime.

WRONG THINKING

Engineers treat a Databricks Runtime upgrade as an infrastructure change that does not affect application behavior. The checkpoint is expected to be forward-compatible with the new runtime version.

WHAT IS ACTUALLY HAPPENING

Spark streaming checkpoints store query plan metadata and state schema information that is serialised using Spark's internal format. Between Spark major versions, the internal serialisation format and query plan representation can change in ways that make old checkpoints unreadable by new Spark versions. A checkpoint written by Spark 3.3 may not be readable by Spark 3.5. This is a known limitation of Structured Streaming checkpoints and is documented in Spark migration guides.

THE FIX

Before performing a Databricks Runtime upgrade on production streaming jobs, check the Spark migration guide for the target version to identify any checkpoint compatibility breaks. Test the upgrade in a staging environment with a copy of the production checkpoint to confirm compatibility. For planned upgrades where the checkpoint is known to be incompatible, schedule a controlled migration: start the new version job with a fresh checkpoint set to a recent source position, allow it to run in parallel with the old version briefly to confirm correctness, then cut over and terminate the old version.

29

CHECKPOINTING

Your checkpoint directory is growing without bound. No cleanup is happening.

THE SITUATION

A streaming job checkpoint directory has grown to 200GB over six months of operation. The checkpoint growth is consuming significant cloud storage costs. No cleanup mechanism has been implemented.

WRONG THINKING

Engineers assume Databricks or Spark manages checkpoint cleanup automatically. Since the checkpoint is managed infrastructure, old checkpoint data is expected to be pruned automatically as newer snapshots are written.

WHAT IS ACTUALLY HAPPENING

Structured Streaming checkpoint directories accumulate log files and state snapshots over time. While Spark does some internal cleanup of old commit and offset log entries, state snapshots and metadata files can accumulate. Spark retains enough checkpoint history to support job recovery but does not aggressively prune historical data. For long-running jobs with frequent micro-batches, the accumulated checkpoint metadata can grow substantially over months.

THE FIX

Monitor checkpoint directory size regularly and implement periodic cleanup. Use the Spark streaming checkpoint cleanup API or manually archive and delete checkpoint subdirectories that are older than the required recovery window. For most streaming jobs a 48 to 72-hour recovery window is sufficient, meaning checkpoint history older than 72 hours can be safely removed. Implement checkpoint cleanup as a separate maintenance job that runs weekly and removes stale checkpoint files beyond the recovery window.

30

CHECKPOINTING

You deleted a checkpoint to fix a stuck stream. The job reprocesses 3 months of data.

THE SITUATION

A streaming job becomes stuck processing the same micro-batch repeatedly. A data engineer deletes the checkpoint to force a clean restart. The job restarts and begins reprocessing 3 months of historical source data before reaching the current position.

WRONG THINKING

Engineers delete the checkpoint as a quick fix for a stuck stream, expecting the job to skip historical data and start from the current position. The checkpoint deletion is treated as a reset rather than a full history wipe.

WHAT IS ACTUALLY HAPPENING

Deleting the checkpoint removes all record of what data has been processed. Without a checkpoint, the streaming job starts from the default position for the source type which for Kafka is the earliest available offset and for file sources is the beginning of all available files. For a source retaining 3 months of history, the job must reprocess all 3 months to reach the current position. There is no way for Spark to know the current position without the checkpoint.

THE FIX

Before deleting a checkpoint, always try less destructive recovery options first. For a stuck stream, check the Spark UI for the specific micro-batch causing the stuck state. If a specific micro-batch is failing, investigate the cause and fix it rather than deleting the checkpoint. If checkpoint deletion is unavoidable, use the source's offset or version metadata to create a new checkpoint that starts from a recent position rather than the beginning of history. For Kafka sources, use `startingOffsets` with specific partition offsets to skip historical data.

05

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 4

Chapter 5 Triggers and Output Modes

Six scenarios covering the correctness and cost failures that appear when the wrong trigger type or output mode is chosen for a streaming workload.

31

TRIGGERS AND OUTPUT MODES

You set trigger once. The job reprocesses previously seen micro-batches.

THE SITUATION

A streaming job is configured with `trigger(once=True)` to process all available data and then stop. On subsequent runs, the job appears to reprocess some micro-batches that were already processed in previous runs. Duplicate rows appear in the output.

WRONG THINKING

Engineers use trigger once expecting idempotent incremental processing. Each run should process only new data since the last run and then stop. Reprocessing of previously processed batches is not anticipated.

WHAT IS ACTUALLY HAPPENING

`trigger(once=True)` was deprecated in favor of `trigger(availableNow=True)`. In some DBR versions, trigger once has subtle behavior differences where the job may not correctly track micro-batch boundaries across separate job runs, leading to overlap in the data processed. Additionally if the checkpoint was written but the micro-batch output was not fully committed due to a partial failure, the next trigger once run may reprocess the uncommitted batch.

THE FIX

Use `trigger(availableNow=True)` instead of `trigger(once=True)` for scheduled streaming batch processing in modern Databricks runtimes. Implement idempotent writes using MERGE or deduplication to ensure that reprocessed micro-batches produce the same output as the original run without creating duplicates. Always use a durable checkpoint with availableNow triggers to ensure each run correctly identifies the starting position.

32

TRIGGERS AND OUTPUT MODES

Your available-now trigger did not stop after processing all available data.

THE SITUATION

A streaming job with `trigger(availableNow=True)` is expected to process all available data and stop automatically. The job continues running after all available data has been processed, consuming cluster resources with no new data to process.

WRONG THINKING

Engineers configure availableNow expecting it to behave like a batch job that terminates when work is complete. The job is expected to stop automatically after processing the backlog.

WHAT IS ACTUALLY HAPPENING

The availableNow trigger processes all data available at the time the streaming query starts and then terminates. If new data arrives during the processing run, this new data may or may not be included depending on when it arrives relative to the trigger's internal snapshot. In some configurations or source types, the streaming job may not correctly detect that all available data has been processed and may remain active indefinitely waiting for a termination condition that never triggers.

THE FIX

Monitor availableNow streaming jobs to confirm they terminate within an expected time window. If a job consistently fails to terminate, add an explicit timeout to the workflow task and investigate whether the source type correctly signals end-of-data to the availableNow trigger. For Delta sources, verify the source table's last version is being read completely. For file sources, confirm that the file discovery mechanism reports all files processed. Consider adding a job-level timeout in the Databricks workflow configuration as a safety net for jobs that should terminate but do not.

33

TRIGGERS AND OUTPUT MODES

You used complete output mode. Your output table grows without bound.

THE SITUATION

A streaming aggregation uses complete output mode. The output table size grows with every micro-batch even though the aggregated metrics being tracked represent a fixed set of dimension values. After several weeks the output table has grown to 10 times the expected size.

WRONG THINKING

Engineers use complete output mode for aggregations expecting it to always show the current complete state of the aggregation. Complete mode is understood to replace the previous output with the current aggregated results.

WHAT IS ACTUALLY HAPPENING

Complete output mode in Structured Streaming rewrites the entire aggregation result set on every micro-batch. For a sink that supports overwrite semantics such as an in-memory table or some file-based sinks, this produces the correct behavior. For Delta Lake in append mode with complete streaming output, Spark appends a new snapshot of the full aggregation result on every micro-batch rather than overwriting the previous result. This is because Delta's default streaming write mode is append. The result is that every micro-batch adds a complete copy of all aggregated rows, causing the table to grow by the full result set size with every trigger.

THE FIX

For streaming aggregations written to Delta using complete output mode, use `foreachBatch` with an explicit overwrite operation rather than relying on the default streaming write semantics. In the foreachBatch function, write the aggregated DataFrame using `df.write.mode('overwrite').saveAsTable(target_table)` to replace the previous result entirely rather than appending to it. This correctly implements complete mode semantics with Delta as the sink.

34

TRIGGERS AND OUTPUT MODES

Update output mode is set. Unchanged rows keep appearing in the output.

THE SITUATION

A streaming aggregation with update output mode writes to a Delta sink. The expectation is that only rows whose aggregated values changed in the current micro-batch are written to the output. After reviewing the output, rows that have not changed since the previous micro-batch are appearing in every micro-batch output.

WRONG THINKING

Engineers choose update output mode to minimize the volume of data written to the sink, expecting only changed rows to be emitted. Rows with unchanged aggregated values should not appear in the output stream.

WHAT IS ACTUALLY HAPPENING

Update output mode emits rows whose aggregated values have changed since the last micro-batch. However if the streaming aggregation includes time-based windows, every micro-batch may update all active window aggregations even if the underlying event counts have not changed for some windows. Spark marks windows as updated whenever they receive new events or when their watermark state changes, even if the aggregate value remains numerically identical. The definition of changed includes any state update, not just numeric value changes.

THE FIX

For use cases where only genuinely different aggregate values should be written, implement a deduplication layer after the streaming aggregation that compares new values against the previously written values and filters out unchanged rows. Use a Delta table as a reference store for the previous values and implement a MERGE that only updates rows where the aggregate value has actually changed. This adds processing overhead but provides true change-only output semantics.

35

TRIGGERS AND OUTPUT MODES

You switched from processingTime trigger to continuous trigger. Exactly once guarantees broke.

THE SITUATION

A streaming job is changed from a processingTime trigger with 30-second intervals to a continuous trigger for lower latency. After the change, duplicate rows begin appearing in the output. The job produces the same event more than once in some cases.

WRONG THINKING

Engineers switch to continuous trigger expecting lower latency with the same correctness guarantees as micro-batch processing. Continuous mode is assumed to be a faster version of micro-batch processing with identical semantics.

WHAT IS ACTUALLY HAPPENING

Continuous processing mode in Spark Structured Streaming uses a fundamentally different execution model from micro-batch mode. Continuous mode provides at-least-once processing guarantees rather than the exactly-once guarantees of micro-batch mode. In continuous mode, Spark processes rows as they arrive without batching them into atomic micro-batches. If a failure occurs mid-stream, some rows may be reprocessed, producing duplicates. The exactly-once guarantee of micro-batch mode comes from the atomicity of each micro-batch commit, which continuous mode does not have.

THE FIX

Use micro-batch processing rather than continuous mode for use cases requiring exactly-once guarantees. For lower latency with exactly-once semantics, reduce the micro-batch interval using `trigger(processingTime='1 second')` or use `trigger(availableNow=True)` on a frequently scheduled job rather than switching to continuous mode. Reserve continuous mode only for use cases that explicitly accept at-least-once semantics and implement idempotent write logic to handle duplicates.

36

TRIGGERS AND OUTPUT MODES

Your fixed interval trigger fires every 60 seconds. Processing takes 90 seconds.

THE SITUATION

A streaming job with a 60-second processingTime trigger is consistently taking 90 seconds to complete each micro-batch. The next trigger fires before the current micro-batch finishes. The job is perpetually behind and the lag grows continuously.

WRONG THINKING

Engineers set a 60-second trigger interval and expect each micro-batch to complete within that interval. When processing takes 90 seconds, the next trigger is expected to wait for the current batch to finish before starting.

WHAT IS ACTUALLY HAPPENING

Structured Streaming with processingTime triggers attempts to fire at the configured interval. If the previous micro-batch has not completed, Spark waits for it to finish before starting the next one. This means the effective trigger interval becomes the processing time rather than the configured interval. For a 90-second processing time with a 60-second interval, micro-batches effectively run back-to-back with no gap. The lag grows because data accumulates during each 90-second processing window but the trigger cannot fire more frequently than one batch at a time.

THE FIX

Optimize the micro-batch processing to complete within the trigger interval by adding cluster capacity, reducing transformation complexity, or increasing partition count to improve parallelism. Alternatively increase the trigger interval to match the actual processing time, which reduces lag accumulation by setting realistic expectations. Monitor the ratio of micro-batch duration to trigger interval as a key streaming health metric. If processing time consistently exceeds the trigger interval by more than 20 percent, the cluster is undersized for the current data volume.

06

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 4

Chapter 6 Stateful Operations

Seven scenarios covering how stateful streaming operations accumulate memory, produce delayed results, and fail in ways that only become visible after extended runtime.

37

STATEFUL OPERATIONS

Your streaming aggregation is correct for fresh data. Wrong for late arriving data.

THE SITUATION

A streaming aggregation with a watermark produces correct results for events that arrive on time. For events that arrive late but within the watermark window, the aggregated values are incorrect. The late events appear to be counted but the running total does not match the sum of on-time and late events.

WRONG THINKING

Engineers verify the aggregation is correct for on-time data and assume late data is handled symmetrically. Since the watermark is configured to include late data, late events are assumed to update the aggregation correctly.

WHAT IS ACTUALLY HAPPENING

Structured Streaming aggregations with watermarks emit partial results for time windows while those windows are still open. When a late event arrives and updates a partially-emitted window, the behavior depends on the output mode. In append mode, windows are only emitted once when they are finalized after the watermark passes. In update mode, windows are re-emitted every time they are updated including when late events arrive. If the downstream consumer accumulates update-mode output by appending each emission, it accumulates both the partial results and the updated results rather than replacing the partial with the updated.

THE FIX

For aggregations that must handle late data correctly, use append output mode which emits each window only once after it is finalized. If update mode is required, ensure the downstream sink supports upsert semantics so late data updates replace rather than duplicate previous emissions. Use Delta with MERGE in foreachBatch to implement correct upsert behavior for streaming aggregations that receive late data updates.

38

STATEFUL OPERATIONS

Your stateful job runs fine for a week then crashes with out of state memory.

THE SITUATION

A streaming job with stateful deduplication using `dropDuplicates` runs correctly for the first week. After seven days the job crashes with an OOM error on the driver. The memory usage graph shows linear growth from day one.

WRONG THINKING

Engineers implement stateful deduplication and test it for several hours in development where it works correctly. Seven days of continuous operation is not tested before production deployment. Memory growth over extended periods is not anticipated.

WHAT IS ACTUALLY HAPPENING

Structured Streaming `dropDuplicates` maintains state for every unique key value seen to identify and drop future duplicates. Without a watermark configured on the deduplication key, this state grows indefinitely because Spark never knows when it is safe to discard old keys. After seven days of processing, the state contains every unique key seen across all seven days. For a high-cardinality key such as a transaction ID or event UUID, this can accumulate millions of entries.

THE FIX

Always configure a watermark when using `dropDuplicates` in streaming. The watermark tells Spark when it is safe to discard deduplication state for keys that are old enough that no late duplicates are expected. Use `df.withWatermark('event_time', '24 hours').dropDuplicates(['key_column', 'event_time'])` to bound state size based on event time. The watermark duration should match the maximum expected late arrival time for duplicate events. Without this bounded state, stateful deduplication will always eventually exhaust memory.

39

STATEFUL OPERATIONS

You added a window aggregation. The results are emitted 2 windows late.

THE SITUATION

A streaming job adds a 5-minute tumbling window aggregation. Engineers expect results for the 9:00 to 9:05 window to appear in the output shortly after 9:05. Instead results appear at approximately 9:15, consistently 2 windows after the window closes.

WRONG THINKING

Engineers expect window results to be emitted as soon as the window closes. A window that ends at 9:05 should produce output at approximately 9:05 when the watermark passes that time.

WHAT IS ACTUALLY HAPPENING

The watermark must advance past the window end time before Spark finalizes and emits the window result in append output mode. The watermark advances based on the maximum observed event time minus the watermark delay. If the watermark delay is 10 minutes, the watermark reaches 9:05 only when events with timestamps of 9:15 or later arrive. For streams where events arrive close to real time, this means waiting approximately 10 minutes after 9:05 before the 9:00 to 9:05 window is emitted. With a 5-minute window and a 10-minute watermark, results are consistently 2 windows behind the current time.

THE FIX

Match the watermark duration to the actual late data arrival pattern rather than using a conservative large watermark. If events rarely arrive more than 2 minutes late, a 2 to 3 minute watermark produces window results 1 to 2 minutes after window close rather than 10 minutes after. Reducing the watermark reduces both the output latency and the amount of state that must be retained. Profile actual late arrival times from production data before setting the watermark for a new streaming job.

40

STATEFUL OPERATIONS

Stream-stream join is producing far more rows than expected.

THE SITUATION

A stream-stream join between two event streams produces output row counts that are 50 times higher than expected. The join condition appears correct and matches the intended logic. The output volume makes downstream processing impractical.

WRONG THINKING

Engineers implement a stream-stream join expecting it to match rows from the two streams on the specified condition. The join is tested with small data samples that produce correct results. At production scale the output volume is unexpected.

WHAT IS ACTUALLY HAPPENING

Stream-stream joins in Structured Streaming maintain state for both streams within the watermark window. Each event in stream A is joined against all events in stream B that fall within the join time constraint. If the join time constraint is not specified or is very wide, events from stream A may match many events from stream B rather than the single expected match. For a many-to-many relationship or when the join condition matches on a non-unique key, each row in stream A can produce multiple output rows by joining with multiple rows from stream B within the retention window.

THE FIX

Add a time-based join constraint that limits the window within which events from both streams can match. Use `df1.join(df2, expr('df1.event_time between df2.event_time - interval 5 minutes and df2.event_time + interval 5 minutes and df1.key = df2.key'))` to restrict matches to events occurring within 5 minutes of each other. This reduces the state retention window and prevents one event from matching many historical events. Verify join key uniqueness in both streams before implementing stream-stream joins to avoid unintended Cartesian-like explosions.

41

STATEFUL OPERATIONS

Your deduplication using dropDuplicates is not removing all duplicates.

THE SITUATION

A streaming job uses `dropDuplicates(['event_id'])` to remove duplicate events from the stream. Downstream analysis shows some duplicate event IDs still appearing in the output. Not all duplicates are being removed.

WRONG THINKING

Engineers implement dropDuplicates on the event ID and assume all duplicates are eliminated. Since event ID uniquely identifies an event, deduplication on this field should produce a completely deduplicated stream.

WHAT IS ACTUALLY HAPPENING

Structured Streaming `dropDuplicates` only removes duplicates within the watermark window. If a duplicate event arrives after the watermark has passed the event's timestamp, the duplicate is treated as a late event that falls outside the deduplication state window. The original event's state has been cleaned up and the late duplicate is treated as a new unique event. Additionally if the streaming job is restarted without a checkpoint, the deduplication state is lost and events that were previously processed are treated as new, potentially creating duplicates from the restart point.

THE FIX

Set the watermark duration for deduplication to cover the maximum time window within which duplicate events can arrive. If duplicates can arrive up to 6 hours after the original event, set a 6-hour watermark. Ensure the checkpoint is preserved across restarts to maintain deduplication state. For compliance-critical deduplication where no duplicates can be tolerated, implement a post-streaming deduplication step against the output Delta table using MERGE or a window function that removes duplicates from the persisted output rather than relying solely on streaming state.

42

STATEFUL OPERATIONS

mapGroupsWithState is holding state for keys that no longer exist in the stream.

THE SITUATION

A streaming job uses `mapGroupsWithState` to maintain per-user session state. After running for several months, the state store contains state for millions of user keys. Most of these users have not sent any events in weeks. Memory usage is dominated by stale state for inactive users.

WRONG THINKING

Engineers implement mapGroupsWithState for session tracking and expect Spark to automatically clean up state for users who have not been active. The state management is treated as automatic.

WHAT IS ACTUALLY HAPPENING

`mapGroupsWithState` retains state for every key that has ever been seen until the state is explicitly removed within the function or a timeout is configured. Without explicit state removal or timeout configuration, state accumulates for all keys indefinitely regardless of how long ago they were last active. A user who logged in once two months ago still has their session state retained in memory because no mechanism has been configured to expire it.

THE FIX

Configure state timeouts in `mapGroupsWithState` using either event time timeouts or processing time timeouts. Call `state.setTimeoutTimestamp(state.getCurrentWatermarkMs() + sessionTimeoutMs)` within the state function to mark keys for cleanup after a defined inactivity period. When the timeout fires, the state function is called with an empty iterator and a timed-out flag allowing explicit state cleanup. Set timeout durations based on the maximum session inactivity period that is considered a closed session for the business use case. For new code on Databricks Runtime 16.2 and above, prefer `transformWithState`, which is the modern arbitrary stateful API and is now Databricks-recommended over `mapGroupsWithState` and `flatMapGroupsWithState` (both classified as legacy operators). transformWithState supports built-in TTL on state values, separates state-eviction logic from input-processing logic, and exposes value, list, and map state primitives directly.

43

STATEFUL OPERATIONS

Your sliding window produces correct counts. The timestamps on the windows are wrong.

THE SITUATION

A 10-minute sliding window with a 5-minute slide produces correct event counts for each window. However the window start and end timestamps in the output are consistently 1 hour behind the actual event times. The counts are right but the time labels are wrong.

WRONG THINKING

Engineers validate the counts in the sliding window output and confirm they are correct. The timestamp columns are not validated during testing. The timestamp discrepancy is discovered by a business user reviewing the output.

WHAT IS ACTUALLY HAPPENING

Sliding window aggregations in Structured Streaming use event time as the basis for window boundaries. If the event timestamp column has a timezone issue such as being stored in UTC but interpreted as local time, the window boundaries are calculated in the wrong timezone. A window covering 9:00 to 9:10 UTC would be labeled as 8:00 to 8:10 if the timezone is interpreted as one hour behind UTC. The counts are correct because the same events fall in the same windows regardless of the timezone offset, but the labels are wrong.

THE FIX

Verify that the event timestamp column is correctly interpreted in the expected timezone. Use `to_timestamp(col('event_time'), 'format')` with an explicit timezone conversion when reading event timestamps to ensure consistent timezone interpretation. Always include timezone-aware timestamps in streaming output so downstream consumers can interpret times correctly. Add a unit test that validates both the count and the timestamp boundaries of sliding window output against known input data.

07

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 4

Chapter 7 Streaming to Delta

Seven scenarios covering the file accumulation, concurrency, and latency patterns specific to Delta Lake as a streaming sink and the maintenance requirements that differ from batch Delta writes.

44

STREAMING TO DELTA

Your streaming job writes to Delta. Small files are accumulating rapidly.

THE SITUATION

A Structured Streaming job writes micro-batches to a Delta table every 30 seconds. After 24 hours the table has 5,760 files from 2,880 micro-batches each producing 2 files. Read performance is degrading as file count grows.

WRONG THINKING

Engineers implement streaming writes to Delta and validate that data is written correctly. File accumulation is not monitored initially because the table is small and performs adequately at first.

WHAT IS ACTUALLY HAPPENING

Every micro-batch write to Delta creates new Parquet files regardless of how few rows the batch contains. A streaming job writing every 30 seconds generates 2 files per minute and 2,880 files per day from write operations alone. Without periodic OPTIMIZE runs, file count grows continuously. At tens of thousands of files, the Delta transaction log overhead and file listing costs for read operations degrade query performance significantly.

THE FIX

Enable optimized writes for streaming Delta sinks by setting `spark.databricks.delta.optimizeWrite.enabled = true` which automatically coalesces small micro-batch writes into larger files. Enable auto compaction with `spark.databricks.delta.autoCompact.enabled = true` which applies incremental background optimization without conflicting with streaming writes. If neither is available or sufficient, run a separate periodic OPTIMIZE job targeting recently written partitions rather than full table OPTIMIZE to minimize conflict with ongoing streaming writes.

45

STREAMING TO DELTA

You write a stream to Delta. Concurrent batch reads see partial micro-batches.

THE SITUATION

A streaming job writes to a Delta table. A batch reporting job reads from the same table while the streaming job is active. Some batch reads return rows from an incomplete micro-batch where only some of the rows have been written, producing inconsistent aggregated results.

WRONG THINKING

Engineers assume Delta Lake's ACID guarantees ensure batch readers always see a consistent view of the table regardless of concurrent streaming writes. A reader should either see a complete micro-batch or not see it at all.

WHAT IS ACTUALLY HAPPENING

Delta Lake provides snapshot isolation for reads. A reader opening a query sees the table state at the version that existed when the query began. However if a streaming job is in the middle of writing a micro-batch when the batch reader starts, the reader sees the version before the micro-batch commit. Once the micro-batch commits, subsequent read queries see the new version. The issue arises when the batch reader is part of a long-running transaction or uses cached metadata from before the streaming commit. In these cases, the batch reader may observe state that reflects partial micro-batch data if Delta's snapshot isolation is not correctly applied.

THE FIX

Ensure batch readers use fresh metadata by avoiding query-level caching of Delta table metadata. Use `spark.sql('REFRESH TABLE tablename')` before batch reads that require current data. For downstream reports that must see a consistent state, design them to read Delta table versions explicitly using time travel to select a stable version rather than reading the live table head which may be mid-write during streaming operations.

46

STREAMING TO DELTA

Your stream writes to a partitioned Delta table. One partition gets all the data.

THE SITUATION

A streaming job writes to a Delta table partitioned by date. All micro-batches are writing to today's date partition. The current date partition has thousands of small files while historical date partitions have a normal file count. Write and read performance for the current date partition is poor.

WRONG THINKING

Engineers partition a streaming Delta sink by date expecting the partition structure to organize data and improve query performance. The specific characteristics of streaming writes to an active partition are not considered.

WHAT IS ACTUALLY HAPPENING

When streaming writes all go to the same active partition such as today's date, every micro-batch adds new files to that partition. The active partition accumulates files at the rate of micro-batch frequency. With 30-second micro-batches, the active partition gains 2 files per minute and 2,880 files per day. OPTIMIZE cannot run on the active partition without conflicting with the ongoing streaming writes. The most recent data, which is typically the most queried, has the worst read performance because it is the least compacted partition.

THE FIX

Run OPTIMIZE on the previous day's partition daily after midnight when streaming writes have moved to the new day's partition. At this point the previous day's partition is no longer receiving streaming writes and OPTIMIZE can run without conflicts. Use a daily maintenance workflow that runs `OPTIMIZE tablename WHERE date = yesterday` to compact each day's partition once it is complete. This ensures historical partitions are well-optimized while accepting that the current active partition will have many small files until the day rolls over.

47

STREAMING TO DELTA

Streaming writes to Delta are slow. Batch writes to the same table are fast.

THE SITUATION

A Structured Streaming job writing to a Delta table achieves 10,000 rows per second. A batch job writing the same data volume achieves 200,000 rows per second. The streaming throughput is 20 times lower than expected given the same cluster and data.

WRONG THINKING

Engineers compare streaming and batch performance and attribute the gap to streaming overhead. Some overhead is expected for streaming but a 20x gap seems excessive. The specific sources of streaming overhead are not investigated.

WHAT IS ACTUALLY HAPPENING

Streaming Delta writes have overhead that batch writes do not. Each micro-batch is an independent Delta transaction that must read the transaction log to check for conflicts, write new data files, and commit a new transaction log entry. For frequent small micro-batches, the transaction overhead is paid many times per second. Batch writes pay this overhead once per write operation regardless of data volume. Additionally streaming jobs maintain checkpoints and state which add I/O overhead. For high-throughput streaming, the per-transaction overhead of Delta commits at high frequency is a significant fraction of total processing time.

THE FIX

Increase micro-batch interval to reduce transaction frequency, accepting higher latency in exchange for higher throughput. Use `trigger(processingTime='30 seconds')` or longer to batch more data into each transaction. Enable optimized writes to reduce the number of files per transaction. For very high-throughput streaming scenarios, consider whether a streaming architecture is the right choice or whether frequent mini-batch processing with Delta would provide better throughput with acceptable latency.

48

STREAMING TO DELTA

You enabled CDF on a streaming target table. Consumer lag increased significantly.

THE SITUATION

Change Data Feed is enabled on a Delta table used as a streaming sink. After enabling CDF, a downstream streaming consumer that reads from this table using CDF experiences a 3x increase in processing lag. The consumer cannot keep up with the rate of new CDF records.

WRONG THINKING

Engineers enable CDF on the streaming sink to allow downstream consumers to track changes. Since CDF adds metadata rather than changing the underlying data, the impact on downstream consumer performance is not anticipated.

WHAT IS ACTUALLY HAPPENING

CDF generates additional records for every insert, update, and delete operation on the table. For a streaming sink receiving high-frequency micro-batch writes, CDF generates a continuous stream of insert records in addition to the data records. The downstream CDF consumer reads both the data records and the CDF metadata records, effectively doubling the volume it must process. For update-heavy workloads, CDF generates both preimage and postimage records which can triple the data volume a consumer must process. The CDF overhead scales with the write frequency of the source table.

THE FIX

Profile the CDF record volume against the data record volume before enabling CDF on high-frequency streaming sinks. If CDF adds more than 50 percent overhead, evaluate whether downstream consumers can be redesigned to read the data table directly rather than through CDF. For consumers that genuinely need change tracking, ensure they have sufficient cluster resources to handle the increased CDF volume. Consider whether a lower-frequency streaming trigger for the sink table would reduce CDF volume to a manageable level.

49

STREAMING TO DELTA

Your stream to Delta uses foreachBatch. Exactly once semantics are broken.

THE SITUATION

A streaming job uses `foreachBatch` to write to Delta with custom logic. After a micro-batch failure and retry, duplicate rows appear in the output. The foreachBatch implementation was expected to provide exactly-once semantics.

WRONG THINKING

Engineers use foreachBatch with Delta writes expecting the combination to provide exactly-once semantics automatically. Since Delta is transactional and foreachBatch allows custom write logic, the output is assumed to be correct after any failure and retry.

WHAT IS ACTUALLY HAPPENING

foreachBatch provides the batch DataFrame and batch ID to a user-defined function. Exactly-once semantics must be implemented explicitly within this function using the batch ID. If the foreachBatch function performs a simple append write without using the batch ID as an idempotency key, a retried batch will append its rows again, creating duplicates. Delta does not automatically deduplicate foreachBatch writes. The responsibility for idempotency lies with the foreachBatch implementation.

THE FIX

Implement idempotent writes in foreachBatch using the batch ID. Use MERGE with the batch ID as part of the match condition, or use Delta's idempotent write feature by passing the batch ID as the app ID and transaction version to the Delta write operation. This ensures that if a batch is retried with the same batch ID, the write is recognized as a duplicate and skipped. Always test foreachBatch logic with simulated batch retries to verify that retried batches do not produce duplicate output.

50

STREAMING TO DELTA

Your streaming job writes correctly. The downstream Delta table shows stale data.

THE SITUATION

A streaming job writes to Delta table A which is the source for a batch pipeline that writes to Delta table B. A downstream report reads from table B. Despite the streaming job running continuously and writing fresh data to table A, the report shows data that is hours old. The batch pipeline appears to be running but table B is not updating.

WRONG THINKING

Engineers confirm the streaming job is writing to table A and assume data flows through to the report automatically. The batch pipeline is running so table B should be current. The stale data is assumed to be a caching issue.

WHAT IS ACTUALLY HAPPENING

The batch pipeline reads from table A and writes to table B. If the batch pipeline is configured to read a specific Delta version or timestamp of table A rather than the latest version, it reads the same historical snapshot on every run. This can happen if the batch pipeline uses time travel syntax accidentally, if it reads a cached version of the metadata rather than refreshing, or if it is reading from a Delta table snapshot created by a view or external reference that was materialized at a specific version. The streaming job writes to the latest version of table A but the batch pipeline never reads beyond its fixed starting version.

THE FIX

Verify the batch pipeline reads the latest version of table A by checking the query for any time travel clauses such as VERSION AS OF or TIMESTAMP AS OF. Remove any time travel references that should not be fixed. Add a metadata check at the start of the batch pipeline that logs the Delta table version being read and confirms it is recent. Set up monitoring that alerts when the version of table A being read by the batch pipeline is more than N versions or M minutes behind the current version.

CONCLUSION

Streaming problems hide behind a running status.

Across 50 scenarios the pattern repeats. Streaming jobs run without errors while dropping late events, accumulating unbounded state, writing duplicates, or falling progressively further behind the input rate. The running status is not a health check. It is a process status. Monitoring streaming health requires looking at input rate versus processing rate, watermark advancement, state size, checkpoint size, and output table freshness separately, because a job can be running successfully on every one of those signals while still being broken on the others.

ALSO IN THE SERIES

Clusters and Compute

> Your job failed in 30 seconds. The cluster spent 28 of them starting up.
>
> You enabled Spot VMs to cut cost. Jobs now randomly fail at 9am.

Delta Lake

> You ran OPTIMIZE. Your queries got slower.
>
> Your MERGE is creating duplicate rows despite a unique match condition.

Workflows and Orchestration

> Your workflow job shows success. One task silently produced no output.
>
> Your DLT expectations are failing. The pipeline succeeds anyway.

And more. Download the full series free at **ssanjaychandra.com**

A NOTE ON THIS RELEASE

This is Book 4 in the Azure Databricks Engineering Patterns series. Most of the scenarios are streaming and Auto Loader mechanics that apply on any cloud, with Azure-specific context introduced where the platform genuinely changes the answer: Event Grid plus Storage Queue as the Auto Loader file notification mechanism on ADLS Gen2, the FlushWithClose event semantics that determine when a file is considered ready to ingest, and abfss paths on ADLS Gen2 as the only durable home for streaming checkpoints. Databricks and Azure both evolve quickly and some guidance may need updating as the platforms mature. If you spot a technical inaccuracy or a scenario that deserves deeper treatment, your feedback is genuinely welcome.

ACKNOWLEDGEMENTS

Claude (Anthropic) was used as a writing assistant while putting this book together.

CONNECT

🌐 www.ssanjaychandra.com